CHALLENGE

03

Predict Anything

If it can go wrong, predict it first.

POWERED BY

Open Track

Open hardware track

Hacktivate 2026 · 24-hour Hardware + AI Hackathon · Theme: Predictive Systems
14 Oct 2026, 11:00 AM to 15 Oct 2026, 11:00 AM · Pravrutti 2.0 (National Tech Fest)

Goals and Motivation

Meet Manjunath

Manjunath runs a small machining workshop in a shed off one of Peenya Industrial Area's inner roads. He has two workers, one lathe and an order for a few hundred precision bushes due to a pump manufacturer on Friday. It is Wednesday afternoon, and the lathe has been running since 8 in the morning.

For the last two days the machine has sounded slightly different. The note of the motor is a little rougher; the bed feels a little warmer under his palm at the end of the day; the finish on the parts needs one more pass than usual. His operator noticed it too, and so did he. Neither of them could say whether it meant "fine for another month" or "stop now".

At 3:40 PM the motor stops with a smell of burnt insulation. The rewinding shop can take it on Saturday. The order is late, the customer calls someone else next time, and the two workers spend Thursday sweeping the floor.

This is not a problem of missing signals. The vibration changed, the temperature crept up, the sound shifted — days before the failure. What was missing was something that **listened to those signals, recognised the trend and said "this motor is likely to fail within the next few days" while there was still time to act.**

Manjunath is not the exception, he is the rule. Peenya is one of the biggest industrial areas in Asia, home to thousands of small and micro units like his, and most of them run on a handful of machines with no maintenance team and no condition-monitoring system. And the factory floor is only one place. The kirana shop that runs out of milk on a Sunday morning, the vaccine box whose temperature drifts on the way to a rural clinic, the bus stop that becomes dangerously crowded every evening — all share the same pattern: the warning signs were there, and nobody was listening.

This challenge asks you to build the thing that listens.

Three Failures. One Challenge.

Across homes, shops, factories and cities, problems keep arriving as surprises for three reasons:

FIX IT WHEN IT BREAKS	Most small operations run on breakdown maintenance and gut feeling. Machines are repaired after they fail, shelves are restocked after they empty, and the cost of the surprise — lost orders, spoiled goods, idle workers — is accepted as normal.
PREDICTION IS PRICED FOR THE BIG PLAYERS	Condition monitoring, demand forecasting and smart sensing exist, but they are typically sold as systems for large plants and chains. The small workshop, the corner shop and the local clinic are left out.
DATA WITHOUT FORESIGHT	Where sensors do exist, they mostly display the present: a temperature reading, a stock count, a dashboard of numbers. Someone still has to watch it, interpret it and guess what happens next.

Your Challenge

Build any hardware product that sees a problem coming and acts before it happens — at home, in a company, on a factory floor or in a city.

This is the open track. You choose the problem, the place and the person. The only constants: it is real hardware, it makes a real prediction, and it does something useful with that prediction before the problem arrives.

A forecast, not a gauge

Your product must work as an early-warning system, not a meter. A gauge tells you the temperature is 38°C; a forecast tells you the cold box will cross the safe limit in about 25 minutes at this rate, and suggests what to do.

Lead time is the value. The earlier and more reliably you can see it coming, the more your product is worth to the person who has to act.

The Rules

- **Any hardware allowed.** Any microcontroller, SBC, sensor or actuator you can run safely. Teams bring their own hardware beyond any shared kit.
- **A real prediction** — time-to-event, an early-rising risk score or a learned pattern. Threshold alarms don't count.
- **Build for someone real.** Talk to the person who would use it.

Building Blocks

Most predictive products are built from these four pieces. They are building blocks, not a prescribed product.

01 Signal Capture

Find the signals that change before the problem does: vibration, sound, current draw (on a safe low-voltage stand-in), temperature trends, weight or count of stock, footfall, door openings. Sample them reliably and log them, because your model is only as good as the data it has seen.

02 Learning Normal

Every machine, shop and street has its own normal. Learn it — a baseline vibration spectrum, a typical sales curve for each weekday, a usual evening crowd — so you can tell a real drift from everyday noise.

03 The Forecast

Turn the drift into a statement about the future: "bearing wear rising, likely failure within days", "milk will run out around 9 AM tomorrow", "cold box will cross 8°C in about 25 minutes". Show the confidence, not just the number.

Required: A real prediction, as defined in The Prediction Requirement below.

04 Act Before It Happens

A forecast nobody acts on is just a better gauge. Alert the right person in the right way (a light on the machine, a message to the owner, a voice prompt), slow something down, trigger a reorder list, or reroute people — whatever fits your user.

Stretch Goals

- **Explainable forecasts** — the product says why it thinks the problem is coming ("vibration at the motor's running frequency has doubled since Monday").
- **Learns on the device** — no cloud needed; works in a shed with patchy connectivity.
- **Cheap enough to matter** — a bill of materials a small workshop or shop could actually afford.
- **Voice** — speak the warning with ElevenLabs (credits for every team).

Directions You Could Explore

The problem is open. Some starting points:

- **Machine failure from vibration or sound** — a lathe, pump, fan or compressor that sounds different days before it fails.
- **A warehouse or shop running low** — predict the stock-out from the pattern of sales, not the empty shelf.
- **A cold chain about to break** — a vaccine box, dairy can or vegetable crate drifting towards spoilage.
- **A crowded city spot** — a bus stop, metro gate or temple queue building towards unsafe density.
- **Home and campus** — a water tank about to run dry, a lab fridge drifting, a plant about to need water.
- **Anything else** — if it can go wrong, predict it first.

The Prediction Requirement

This is a Predictive Systems hackathon, and this track lives or dies on the quality of the forecast.

Your device must make a **real prediction**. That means it forecasts something that has not happened yet, in one of these forms:

- **Time-to-event** — "about 40 seconds until this happens", updated as the situation changes.
- **An early-rising risk score** — a likelihood that climbs as precursors build up, well before the event itself.
- **A learned pattern** — the device learns what normal looks like for this person, place or machine, and flags the drift away from it before it becomes a problem.

Threshold alarms do not count. "Beep when the distance drops below 50 cm" or "buzz when the temperature crosses 40°C" is a reaction, not a prediction. A real forecast says what is coming and how soon, before any threshold is crossed.

Examples of what a real prediction looks like for this challenge:

- A vibration monitor on a small DC motor that learns its healthy signature and shows a rising wear score as an imbalance weight is added — before any threshold is crossed.
- A shelf sensor that learns the daily sales curve and predicts the hour the last packet will go, updating as items are taken faster.
- A cold-box logger that projects time-to-unsafe from the warming rate and updates when the lid is opened.

Required: Show what your product predicts, how far ahead and how sure it is, and show it working live in your demo.

Build for Someone Real

Pick your own problem, but build it for a real person, not a generic "user". A workshop owner in Peenya, the canteen manager on campus, a pharmacist, a kirana shopkeeper, a bus conductor — someone who lives with the problem you chose.

Required: Talk to at least one real user before or during the hackathon (in person or on a call). Bring to judging: **one direct quote** from them, **one insight** you did not have before, and **one design change** that conversation caused.

The Demo Requirement

Hacktivate is judged on what works in front of the judges. There are no slides. Your demo is the device, your KiCad design and the person you built it for.

- **Live, no slides.** You present the working hardware on the table. A phone or laptop may show your sensor readings, forecast plot or serial monitor, but not a pitch deck.

- **KiCad circuit.** Show the schematic (and PCB layout, if you got that far) for the circuit you actually built. It should match the hardware on the table.
- **Working hardware.** A simulation, video or Wokwi model does not count as the demo.
- **Show it working.** Run a live demo of your product doing its job: sensing, making its prediction and acting on it.

Required: All of the above, in a demo of about five minutes, followed by questions.

Resources

Hardware

- **Any hardware is allowed.** Microcontrollers (ESP32, Arduino, STM32), SBCs (Raspberry Pi and similar) and any sensors or actuators.
- **Bring your own.** Teams bring all their own hardware. The PCB Cupid kit is reserved for Challenge 01 teams.
- **Provided by organisers:** ElevenLabs credits for every team.
- **Ideas, not requirements** — sensors teams often use: accelerometers (MPU6050 and similar), microphones, temperature/humidity, load cells, ultrasonic/IR, light, hall sensors, current sensing on low-voltage circuits.
- **Safe stand-ins** — model a factory motor with a small DC motor or fan, a cold chain with an insulated box and ice, a shelf with a few objects on a load cell.

Tools and Methods

- **KiCad** for the schematic and PCB layout.
- **Simple, explainable methods first** — moving averages, rate-of-change, linear regression over a sliding window, FFT for vibration and sound, anomaly scores against a learned baseline.
- **Tiny ML** — TensorFlow Lite for Microcontrollers or Edge Impulse for on-device models.
- **ElevenLabs** credits for every team, if your product speaks.

What Makes a Strong Submission

Strong Submissions...	Weak Submissions...
— Show a real forecast. The predicted time-to-failure or risk score leads the problem and updates sensibly as conditions change.	— Build a threshold alarm — "buzzer at 40°C" — and call it predictive.
— Learn normal first. The product knows what healthy looks like and detects drift from it.	— Hard-code limits copied from a datasheet with no learning from real data.
— Act on the prediction. The right person gets the right warning, early enough to do something.	— Stop at a dashboard of numbers that someone has to stare at.
— Build for a real place. A Peenya workshop, a kirana shop, a campus canteen — with a quote from the person who runs it.	— Solve a vague "Industry 4.0" problem for no one in particular.
— Be honest about limits — lead time, confidence and what your stand-in can and can't show.	— Overclaim accuracy that the demo can't back up.

Evaluation Criteria

Every challenge is judged out of 20 marks. Creativity breaks ties.

Criterion	Marks	Weight	What judges look for
KiCad design	5	25%	A clean schematic for the hardware you built, matching the device on the table; sensible power and signal conditioning.
Hardware concept and working (incl. prediction quality)	7	35%	Is the prediction real and well-reasoned? Does the product act on it?
User understanding	3	15%	A real person, a real quote, a genuine insight and a design change that came from them.
Live demo	5	25%	Live, no slides; clear in about five minutes; the product works in front of the judges.

Ground Rules

- **Safe stand-ins only.** Model real machinery with safe, small-scale stand-ins; never connect your build to real industrial equipment.
- **Assistive and early-warning aids only.** No medical diagnosis claims. "Your posture is drifting" is fine; "you have a spinal condition" is not.

Why This Matters

Big companies have been predicting failures, shortages and spoilage for years, with expensive sensors, software and teams of analysts. The people who are hurt most by surprises — the small workshop with one machine, the shop with one day of stock, the clinic with one fridge — have had none of it. For them, every surprise is a lost order, wasted stock or a risk to someone's health.

Cheap sensors and small, efficient models have changed what's possible. A forecast that once needed a control room can now run on a board that costs less than a day's lost production. If you can build a product that sees a problem coming and acts in time, for someone who has never had that kind of foresight, you won't just win a hackathon. You'll have built something that places like Peenya could actually use on Monday morning.

Appendix — Contact & Additional Resources

Challenge contact: Ganesh S, Chair, IEEE SB, Ramaiah University of Applied Sciences · voxmastery@ambugo.tech · +91 79753 24769

Web: pravrutti.tech · msruas.ac.in

- On Peenya: Peenya Industrial Area, Bengaluru, is widely described as one of Asia's largest industrial areas (see The Hindu; Wikipedia, "Peenya").
- KiCad: docs.kicad.org · Edge Impulse: docs.edgeimpulse.com · ElevenLabs: elevenlabs.io/docs